Accenture Demo QA

Material de estudo para apresentação — Automação de Testes (UI + API)

Site sob teste: demoqa.com · Stack: Cypress + Cucumber (BDD) + Allure

---

> **Como usar este material:** cada seção tem o *o quê*, o *porquê* e o *como*. A última seção traz perguntas prováveis da banca com respostas — ensaie falar em voz alta. O objetivo não é decorar, é **entender para explicar**.

1. Visão geral do projeto

O projeto automatiza o site DemoQA em dois domínios: **testes de UI** (interface) e **testes de API** (o desafio BookStore). Ambos usam **Cypress** como motor, **Cucumber (Gherkin/BDD em português)** para descrever os cenários em linguagem natural, e **Allure** para o relatório.

Domínio	Pasta	O que cobre
UI	`cypress/`	4 fluxos: Practice Form, Browser Windows, Progress Bar e Sortable (drag and drop)
API	`api-tests/`	Fluxo BookStore E2E + cenários alternativos, com validação de contrato (JSON Schema) e mensagens de erro

Por que BDD/Cucumber?

Os cenários ficam legíveis para qualquer pessoa (QA, PO, dev) porque são escritos em **Gherkin** (`Dado / Quando / Então`). A implementação técnica fica separada nos *step definitions*. Isso aproxima a documentação viva do teste real.

2. Arquitetura

O código é organizado em camadas, para não repetir lógica e facilitar a manutenção:

```
feature (.feature, Gherkin)
   ↓
step definitions  (traduz cada passo em ações)
   ↓
pages / keywords  (regras de domínio: login, livros, formulário...)
   ↓
actions / service (ações base: clicar, preencher, request HTTP, validar)
   ↓
elements / endpoints (seletores CSS / rotas da API)
```

🧩 **Camada de actions (UI):** todo acesso a elemento passa por `cypress/support/actions.js` (clicar, preencher, validar, etc.). Centraliza e padroniza.

🔁 **Steps compartilhados:** navegação comum (home, card, submenu) fica em `common_steps.js`, reutilizada pelos 4 fluxos de UI.

🌐 **baseUrl + endpoints relativos:** a URL base fica na config; os testes usam só `/forms`, `/sortable`, etc.

> ⚠️ **Rodar no Chrome** (`--browser chrome`). O Fluxo 4 (Sortable) usa eventos de mouse reais via **CDP**, que só funcionam no Chrome real — não no Electron headless padrão. Os scripts `test` e `cy:run:chrome` já forçam o Chrome.

3. Fluxos de UI

Fluxo 1 — Practice Form `formulário + upload`

Home → **Forms** → **Practice Form** → preenche tudo com dados aleatórios (faker) → faz upload de um `.txt` → submete → valida o popup → fecha o popup.

> **Desafio:** fechar o popup não funcionava — o clique no botão (mesmo com `force`) não disparava o handler do React, por causa de iframes de anúncios sobrepostos.
> **Solução:** fechar o modal pela tecla **ESC** (o react-bootstrap escuta ESC). Virou a action `pressionarTecla`.

Fluxo 2 — Browser Windows nova janela

Home → **Alerts, Frame & Windows** → **Browser Windows** → clica em **New Window** → valida a mensagem "This is a sample page" → fecha.

> **Desafio:** o Cypress não controla múltiplas abas/janelas nativamente.
> **Solução:** `cy.stub` em `window.open` para capturar a URL que seria aberta, navegar até ela no mesmo contexto e validar o conteúdo.

Fluxo 3 — Progress Bar timing

Home → **Widgets** → **Progress Bar** → Start → para antes dos 25% → valida ≤ 25% → reinicia até 100% → reseta.

> **Dois comportamentos reais descobertos por diagnóstico:**
>
> - A barra sobe ~1% a cada 100ms. Para parar com segurança antes dos 25%, uso **polling** (retry do `should`) e paro numa faixa com folga (abaixo da metade do limite), absorvendo a latência até o clique de Stop.
> - O botão **Reset** (aos 100%) **zera e reinicia** a contagem sozinho. Por isso a validação confirma que o botão Reset some e o valor sai de 100% — em vez de exigir valor estável em 0.

Fluxo 4 — Sortable (drag and drop) o mais difícil

Home → **Interactions** → **Sortable** → **embaralha** a lista → **reordena** em ordem crescente (One → Six) via drag and drop → valida a ordem.

A página usa **react-sortable-hoc** (baseado em eventos de mouse/pointer), notoriamente difícil de automatizar. Progressão das tentativas:

Abordagem	Resultado
`cy.trigger` sintético (mousedown/move/up, pageX/pageY, buttons:1)	❌ **não reordena**
`cypress-real-dnd` (CDP HTML5 drag)	❌ **não cobre libs de mouse-event (por design)**
`cypress-real-events` no Electron headless	❌ **não reordena**
`cypress-real-events` no **Chrome real** + movimentos iniciais **dentro do item**	✅ **drag funciona**
Controle fino da posição de destino (pular N posições)	⚠️ **instável**

> **O que destravou:** o react-sortable-hoc só reconhece o arraste quando os primeiros `realMouseMove` saem **de dentro do próprio item** (com `keepMouseDownButton: "left"`) e **no Chrome real**.
>
> **Como ficou confiável:** em vez de "pular N posições" (instável), uso **trocas adjacentes** (bubble sort): embaralho a lista e reordeno movendo um item uma posição por vez, relendo o DOM entre as trocas. Isso torna o drag **real e visível**.

4. Testes de API — Desafio BookStore

Fluxo do desafio (Swagger em `demoqa.com/swagger`), em Cypress (`cy.request`) com Cucumber. Cada passo valida **status code + contrato (JSON Schema via Ajv) + dados**.

1. **Criar usuário** — `POST /Account/v1/User`
2. **Gerar token** — `POST /Account/v1/GenerateToken`
3. **Confirmar autorização** — `POST /Account/v1/Authorized`
4. **Listar livros** — `GET /BookStore/v1/Books`
5. **Alugar 2 livros** — `POST /BookStore/v1/Books`
6. **Detalhes do usuário com os livros** — `GET /Account/v1/User/{id}`

Cenários alternativos (com validação da mensagem de erro)

Cenário	Esperado
Usuário já existente	`406` · code 1204 · "User exists!"
Senha fraca	`400` · code 1300 · regra de senha
userName / password vazio	`400` · code 1200 · "UserName and Password required."
Token com usuário inexistente	`200` · status "Failed"
Authorized com usuário inexistente	`404` · code 1207 · "User not found!"
Alugar sem token	`401`
Alugar ISBN inexistente	`400` · code 1205 · "ISBN supplied is not available..."

Independência + base limpa (setup/teardown)

> Cada cenário **cria o seu próprio usuário** (nome aleatório, evita colisão) e o **exclui no fim** via `DELETE /Account/v1/User/{id}` num hook `After` do Cucumber — rodando mesmo se o teste falhar.
>
> Resultado: os testes são **independentes** (rodam em qualquer ordem) e **não sujam a base**. O usuário "nasce e morre" dentro do fluxo de cada teste.

Descoberta importante da API

O endpoint `Authorized` retorna `false` para um usuário recém-criado e só passa a `true` **depois** de gerar o token. Isso foi descoberto validando direto na API — por isso o cenário de autorização gera o token antes de confirmar.

5. Método de trabalho (como os problemas foram resolvidos)

- **Diagnosticar antes de corrigir:** inspecionar o estado real do DOM/resposta no momento da falha, em vez de chutar a solução.
- **Testar hipóteses isoladas** em cenários de debug temporários, removidos depois.

- **Parar de repetir o que não funciona:** após falhas, investigar a causa raiz ou alinhar a decisão (ex.: o trade-off do drag).
- **Validar de verdade:** não confiar só no "passou" — ex.: confirmei na API que o usuário realmente foi excluído após o teardown.

6. Como executar

```
# instalar dependências
npm install

# UI (Chrome — necessário para o Fluxo 4)
npm run cy:run:chrome

# API
npm run api:run

# UI + API de uma vez
npm run test:all

# tudo + relatório Allure
npm run test:all:allure
```

O Allure precisa de um runtime Java (JRE/JDK) para gerar o relatório HTML.

7. Perguntas prováveis na apresentação ensaie em voz alta

Por que você escolheu Cypress + Cucumber?

Cypress pela produtividade e bom suporte a UI e API no mesmo projeto. Cucumber porque deixa os cenários legíveis para o time todo (Gherkin em português) e separa o "o quê" (feature) do "como" (steps), aproximando documentação e teste.

Por que o Sortable precisa do Chrome?

A lib react-sortable-hoc só reage a eventos de mouse "reais" (isTrusted). Os eventos sintéticos do Cypress são ignorados. Uso o cypress-real-events, que injeta eventos nativos via CDP — e esse CDP de injeção só funciona confiável no Chrome real, não no Electron headless.

O drag realmente arrasta ou só valida a ordem?

Arrasta de verdade. Antes ele só validava a ordem já entregue (nenhum drag acontecia). Corrigi: agora o fluxo embaralha a lista e a reordena por trocas adjacentes, então o drag é executado e visível. Confirmei por diagnóstico que a lista fica bagunçada antes e volta a crescente no fim.

Como você garante que os testes de API não sujam o banco?

Cada cenário cria seu usuário e o exclui no teardown (hook After, via DELETE). Como rodam com nomes aleatórios e cada um limpa o que criou, são independentes e não deixam resíduo. Validei na API que o usuário some após o delete.

Como funciona a validação de contrato?

Uso JSON Schema com a biblioteca Ajv. Cada resposta é validada contra um schema (campos e tipos esperados). Isso pega mudanças de contrato que uma asserção pontual deixaria passar.

Por que testar múltiplas janelas com stub e não abrir a janela de verdade?

O Cypress roda num único contexto de navegador e não controla uma segunda janela. A prática padrão é interceptar window.open com um stub, capturar a URL e validar o conteúdo navegando até ela — testando o efeito esperado sem depender de um recurso que a ferramenta não suporta.

Qual foi a parte mais difícil e o que você aprendeu?

O drag and drop. Aprendi a diferença entre eventos sintéticos e nativos (isTrusted/CDP), que bibliotecas de drag exigem eventos reais, e a importância de diagnosticar a causa raiz em vez de tentar variações no escuro. Também aprendi a ser transparente sobre limitações e documentá-las.

Para gerar um PDF: abra este arquivo no navegador e use **Imprimir → Salvar como PDF**.